南京大学
NANJING UNIVERSITY

数据结构
Data Structure

许 蕾

xlei@nju.edu.cn

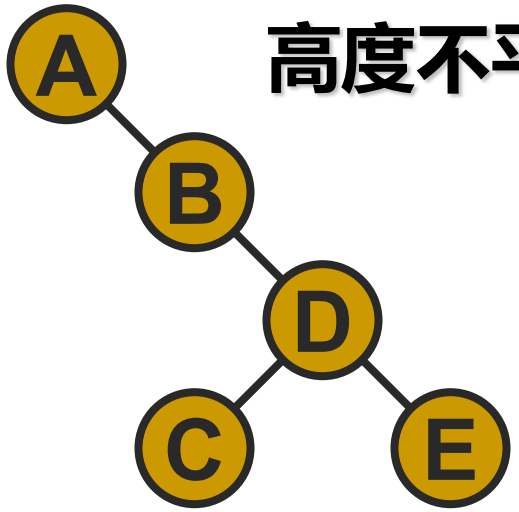
南京大学计算机学院

第七章 搜索结构

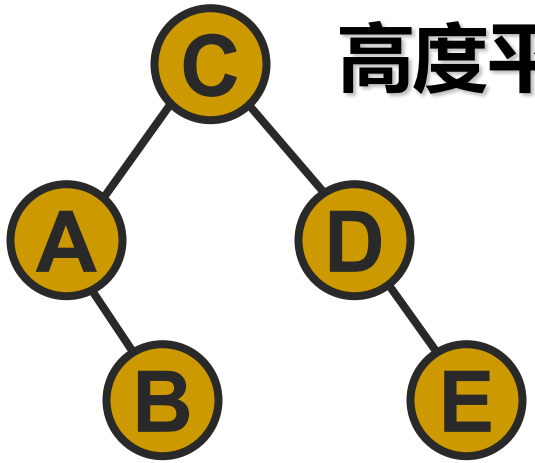
- 静态搜索结构
- 二叉搜索树
- AVL树

7.3 AVL树--- 高度平衡的二叉搜索树

- **AVL 树的定义**　一棵 AVL 树或者是空树，或者是具有下列性质的**二叉搜索树**：
  - 它的左子树和右子树都是 AVL 树，且左子树和右子树的高度之差的绝对值不超过1。



高度不平衡



高度平衡

- 1962年由两位俄罗斯数学家G.M. Adelson-Velsky 和 E.M. Landis在论文“An algorithm for the organization of information”中提出

Thus (5.1) is not true for this function for $l = 1$ and $r = 1$.

Received 22/MAR/62

BIBLIOGRAPHY

[1] C. Miranda, *Equazioni alle derivate parziali di tipo ellittico*, Springer, Berlin, 1955.
[2] S. M. Nikol'skiĭ, Sibirsk. Mat. Ž. 1 (1960), 78.

Translated by:
F. M. Goodspeed

AN ALGORITHM FOR THE ORGANIZATION OF INFORMATION

G. M. ADEL'SON-VEL'SKIĬ AND E. M. LANDIS

In the present article we discuss the organization of information contained in the cells of an automatic calculating machine. A three-address machine will be used for this study.

Statement of the problem. The information enters a machine in sequence from a certain reserve. The information element is contained in a group of cells which are arranged one after the other. A certain number (the information estimate), which is different for different elements, is contained in the information element. The information must be organized in the memory of the machine in such a way that at any moment a very large number of operations is not required to scan the information with the given evaluation and to record the new information element.

An algorithm is proposed in which both the search and the recording are carried out in $C \lg N$ operations, where N is the number of information elements which have entered at a given moment.

A part of the memory of the machine is set aside to store the incoming information. The information elements are arranged there in their order of entry. Moreover, in another part of the memory a "reference board" [1] is formed, each cell of which corresponds to one of the information elements.

The reference board is a dyadic tree (Figure 1*a*): each of its cells has no more than one left cell, and no more than one right cell subordinated to it. Direct subordination induces subordination (partial ordering). In addition, for each cell of the tree, all the cells which are subordinate to a left (right) directly subordinate cell, will be arranged further to the left (right) than the given cell. Moreover, we assume that there is a cell (the head) to which all the others are subordinate. By transitivity, the conception "further to the left" and "further to the right" extends to the aggregate of all the cell pairs, and this aggregate becomes ordered. Thus, a given order of cells in a reference board should coincide with the order of arrangement of the estimates of the corresponding information elements (to be specific, we shall consider the estimates as increasing from left to right).

In the first address of each cell of the reference board, a place is indicated where the corresponding information element is located. The addresses of the cells of the reference board, which are directly subordinate on the left and right respectively to the given cell, are located in the second and third addresses. If a cell has no directly subordinate cells on either side, then there is zero in the corresponding address. The head address is stored in a certain fixed cell l.

Let us call the sequence of the cells of the tree a *chain* in which each previous cell is directly

结点的平衡因子
bf (balance factor)

- 每个结点附加一个数字，给出该结点**右子树的高度减去左子树的高度**所得的高度差，这个数字即为结点的**平衡因子bf**。
- AVL树任一结点的平衡因子只能取 -1, 0, 1。
- 若一个结点的平衡因子的绝对值大于1，则这棵二叉搜索树就失去了平衡，不再是AVL树。
- 如果一棵有 n 个结点的二叉搜索树是高度平衡的，其高度可保持在$O(\log_2 n)$，平均搜索长度也可保持在$O(\log_2 n)$。

AVL树的类定义

```
#include <iostream.h>
#include "stack.h"
template <class K, class E>
struct AVLNode : public BSTNode<K, E> {
                                        //AVL树结点的类定义
   int bf;
   AVLNode() { left = NULL; right = NULL; bf = 0; }
   AVLNode (K k, E x, AVLNode<K, E> *l = NULL,
         AVLNode<K, E> *r = NULL)
      { key = k; data = x;  left = l;  right = r;  bf = 0; }
};
```

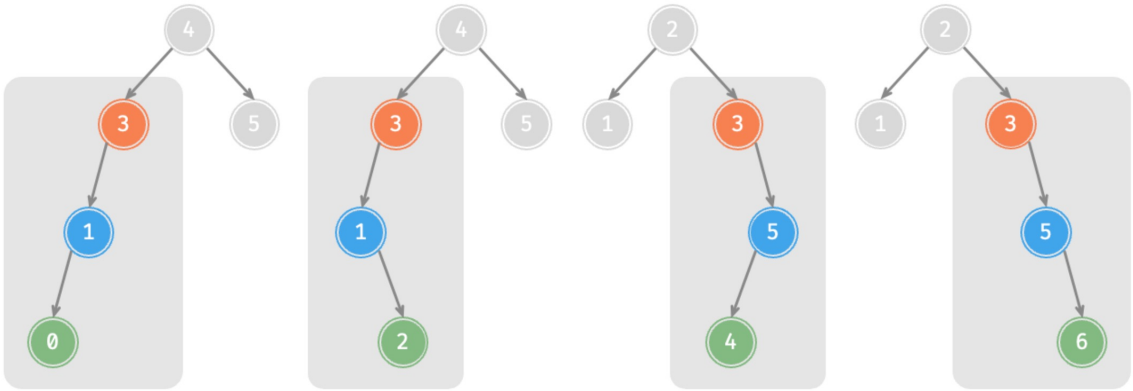
```
template <class K, class E>
class AVLTree : public BST<K, E> {
                    //平衡的二叉搜索树（AVL）类定义
public:
    AVLTree() { root = NULL; }          //构造函数
    AVLTree (K Ref) { RefValue = Ref; root = NULL; }
                                //构造函数：构造非空AVL树
    int Height() const;       //高度
```

```
AVLNode<K, E>* Search (K x, AVLNode<K, E> *& ptr)
                                    const;      //搜索
  bool Insert (K x, E& e1) { return Insert (root, x, e1); }
                                                //插入
  bool Remove (K x, E& e1)
          { return Remove (root, x, e1); }          //删除
  friend istream& operator >> (istream& in,
     AVLTree<K, E>& Tree);                    //重载：输入
  friend ostream& operator << (ostream& out,
     const AVLTree<K, E>& Tree);              //重载：输出
protected:
  int Height (AVLNode<K, E> *ptr) const;
```

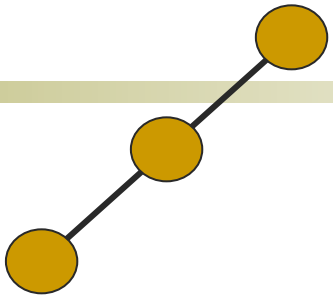
```
    bool Insert (AVLNode<K, E>*& ptr, K x, E& e1);
    bool Remove (AVLNode<K, E>*& ptr, K x, E& e1);
    void RotateL (AVLNode<K, E>*& ptr);    //左单旋
    void RotateR (AVLNode<K, E>*& ptr);    //右单旋
    void RotateLR (AVLNode<K, E>*& ptr);
                                           //先左后右双旋
    void RotateRL (AVLNode<K, E>*& ptr);
                                           //先右后左双旋
};
```

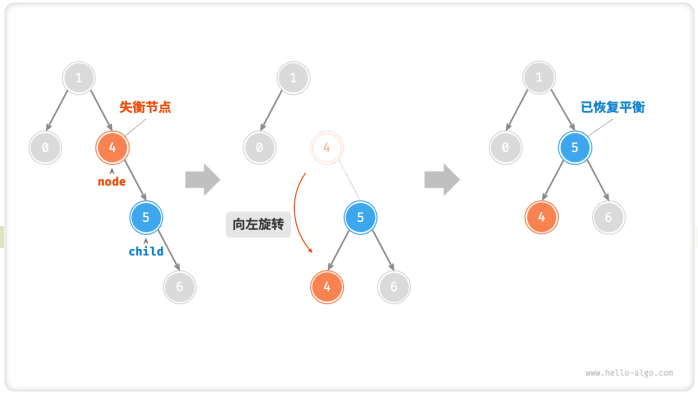
AVL树的平衡化旋转

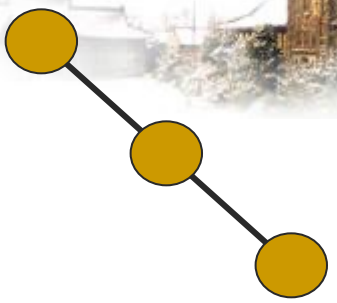
- 如果在一棵平衡的二叉搜索树中插入一个新结点或者删除一个结点后，造成了不平衡。必须调整树的结构，使之平衡化。
- 平衡化旋转有两类：
  - 单旋转 (左单旋转和右单旋转)
  - 双旋转 (先左后右和先右后左)



右旋　先左旋后右旋　先右旋后左旋　左旋

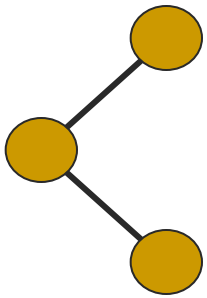


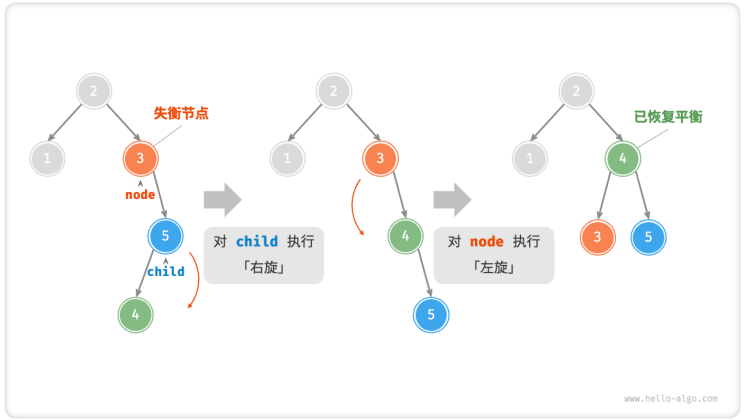


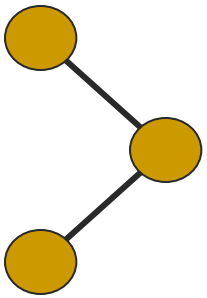


右单旋转

左单旋转





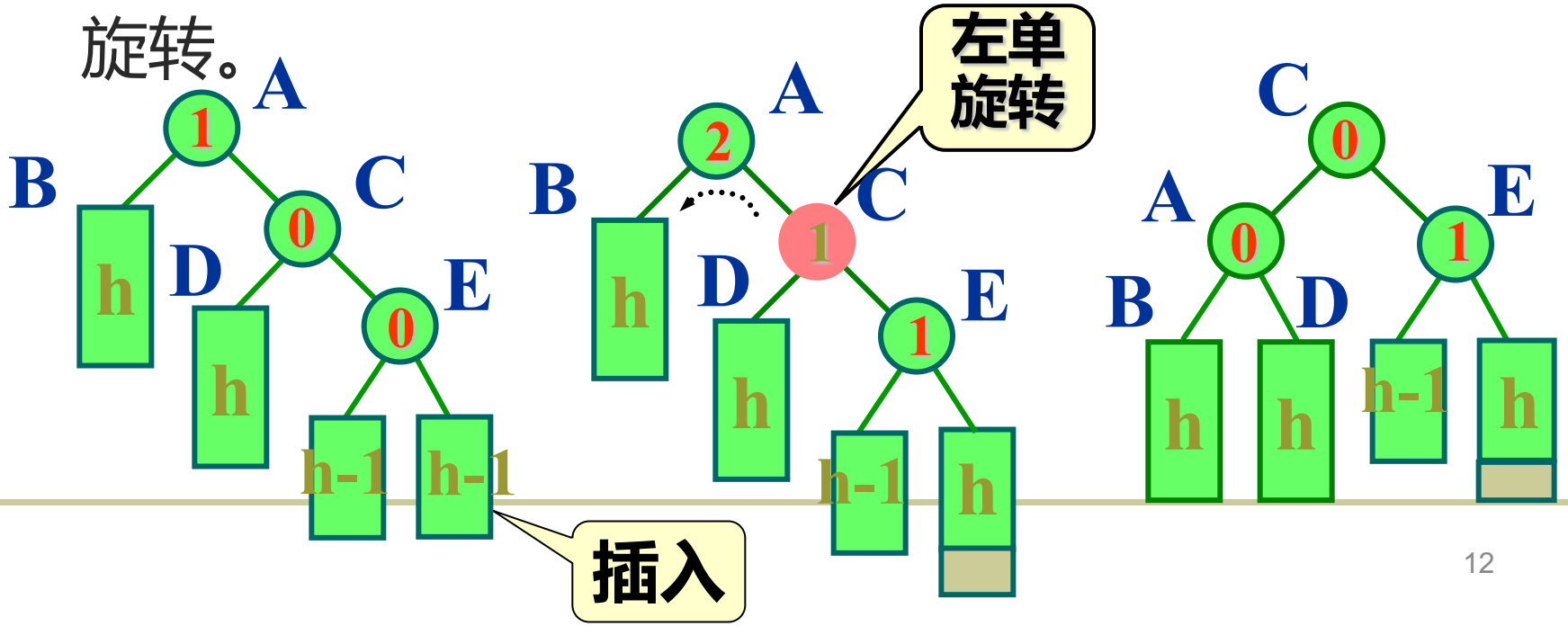


先左后右双旋转

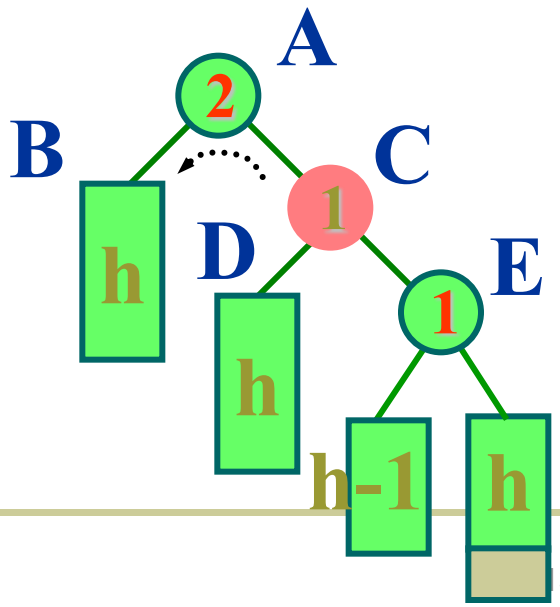
先右后左双旋转

左单旋转 (RotateLeft)

- 在结点A的右子女的右子树E中插入新结点，该子树高度增1导致结点A的平衡因子变成2，出现不平衡。
- 为使树恢复平衡，从A沿插入路径连续取3个结点A、C和E，以结点C为旋转轴，让结点A反时针旋转。

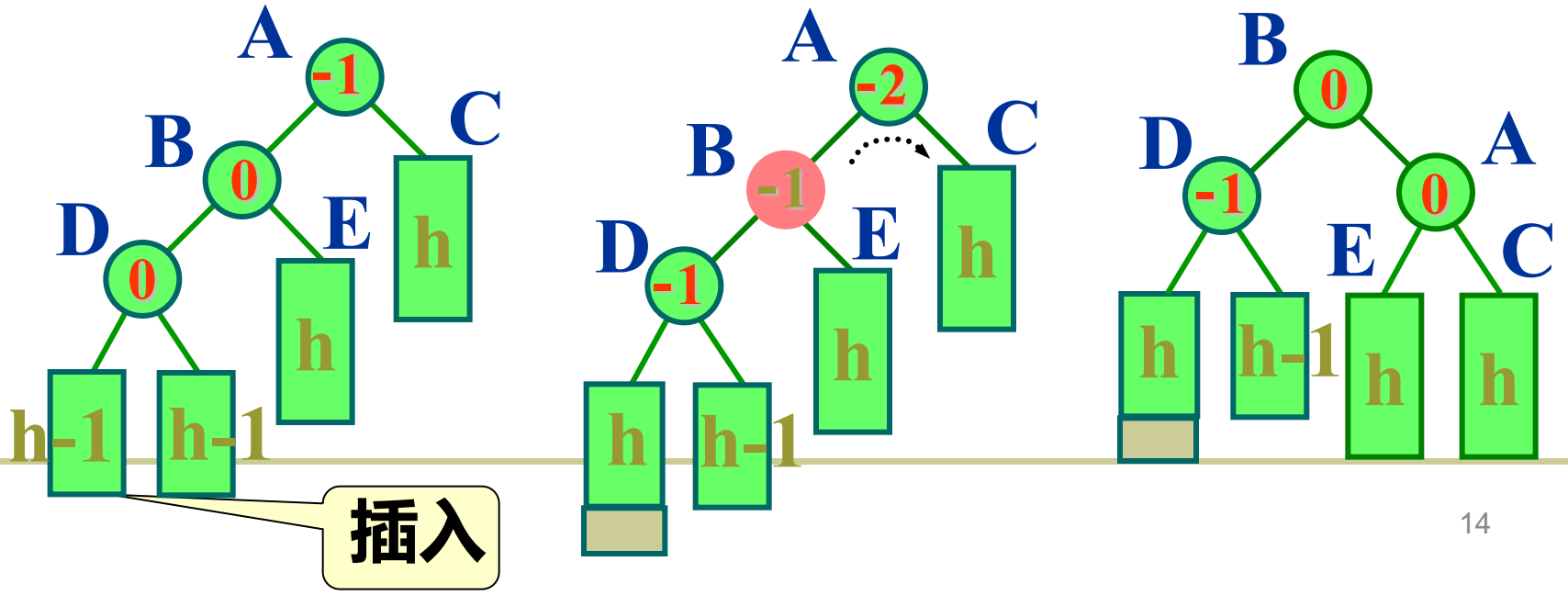


```
template <class K, class E>
void AVLTree<K, E>::RotateL (AVLNode<K, E> *&
    ptr) {
//右子树比左子树高: 做左单旋转后新根在ptr
     AVLNode<K, E> *subL = ptr;
    ptr = subL->right;
    subL->right = ptr->left;
    ptr->left = subL;
    ptr->bf = subL->bf = 0;
};
```

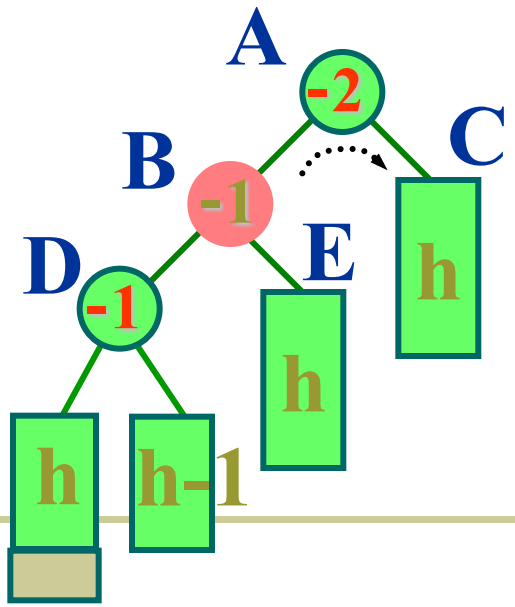


右单旋转 (RotateRight)

- 在结点A的左子女的左子树D上插入新结点使其高度增1导致结点A的平衡因子增到-2，造成不平衡。
- 为使树恢复平衡，从A沿插入路径连续取3 结点A、B和D，以结点B为旋转轴，将结点A顺时针旋转。

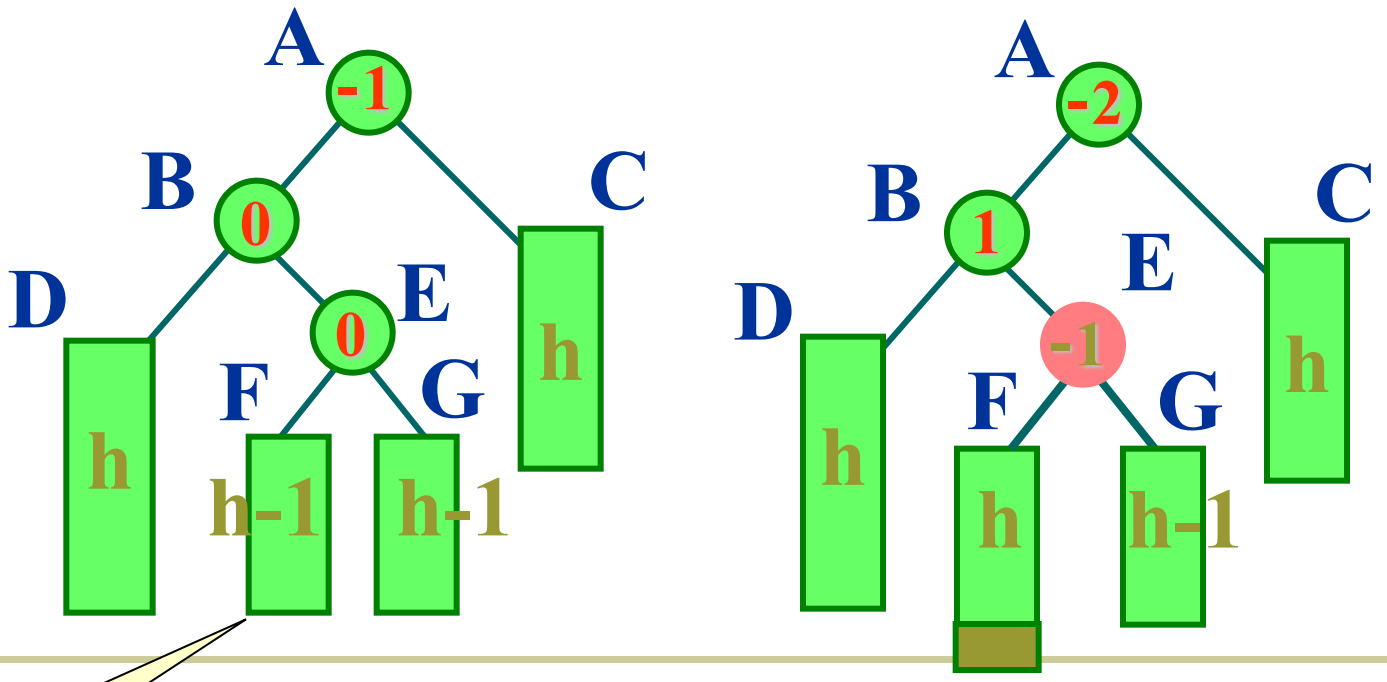


```
template <class K, class E>
void AVLTree<K, E>::RotateR(AVLNode<K, E> *& ptr)
{  //左子树比右子树高, 旋转后新根在ptr要右旋转
   AVLNode<K, E> *subR = ptr;                    //的结点
   ptr = subR->left;
   subR->left = ptr->right;
   ptr->right = subR;
   ptr->bf = subR->bf = 0;
};
```

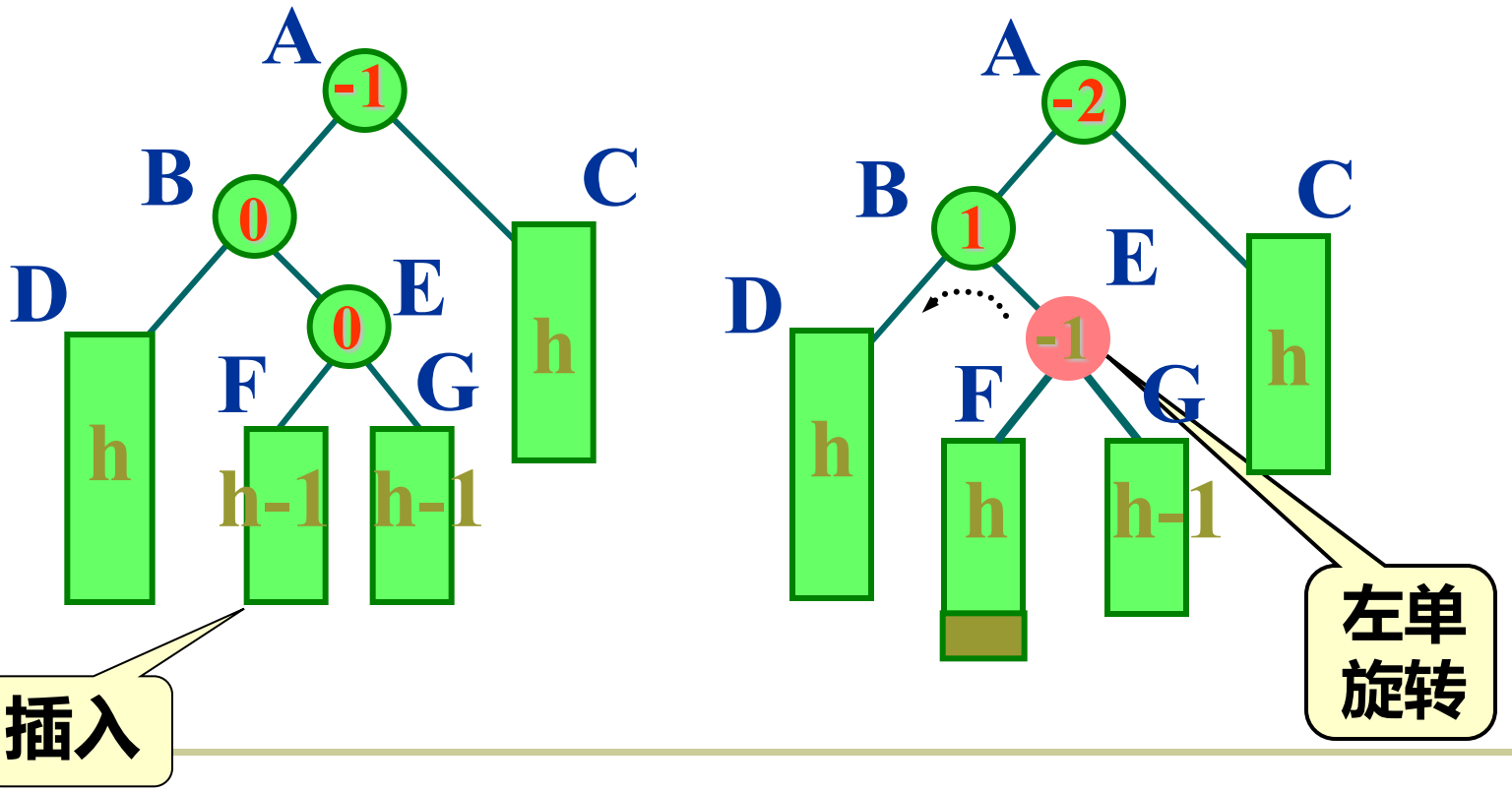


先左后右双旋转 (RotationLeftRight)

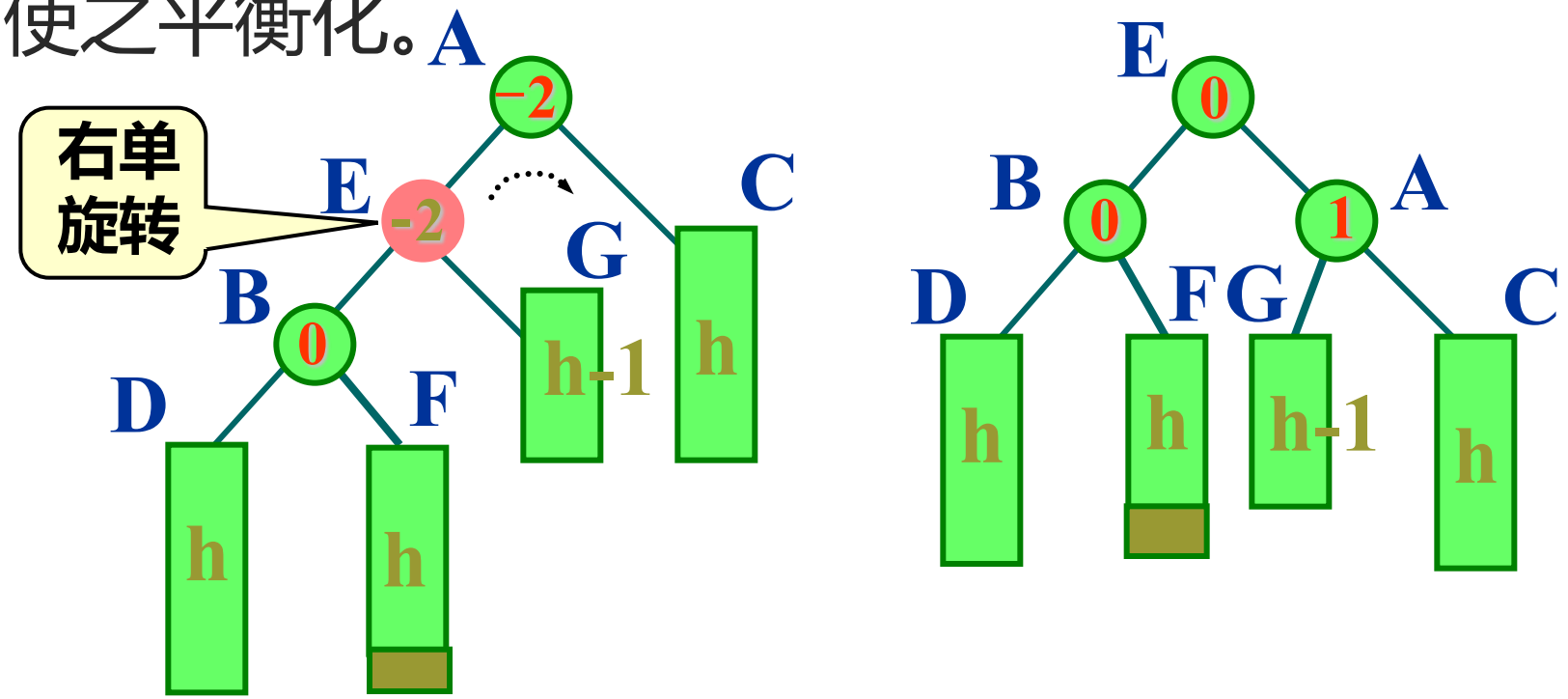
- 在结点A的左子女的右子树中插入新结点，该子树高度增1导致结点A的平衡因子变为-2，造成不平衡。



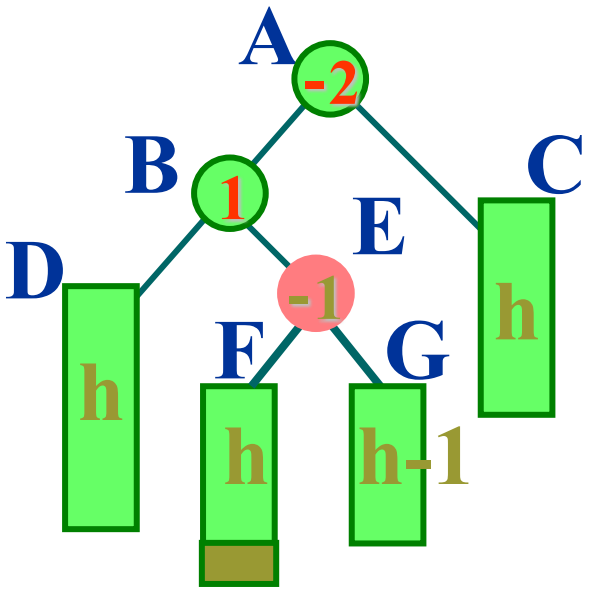
- 以结点E为旋转轴，将结点B反时针旋转，以E代替原来B的位置。

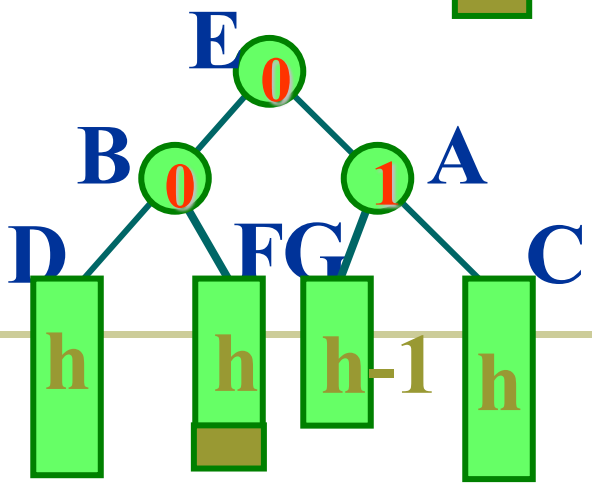


- 再以结点E为旋转轴，将结点A顺时针旋转。使之平衡化。



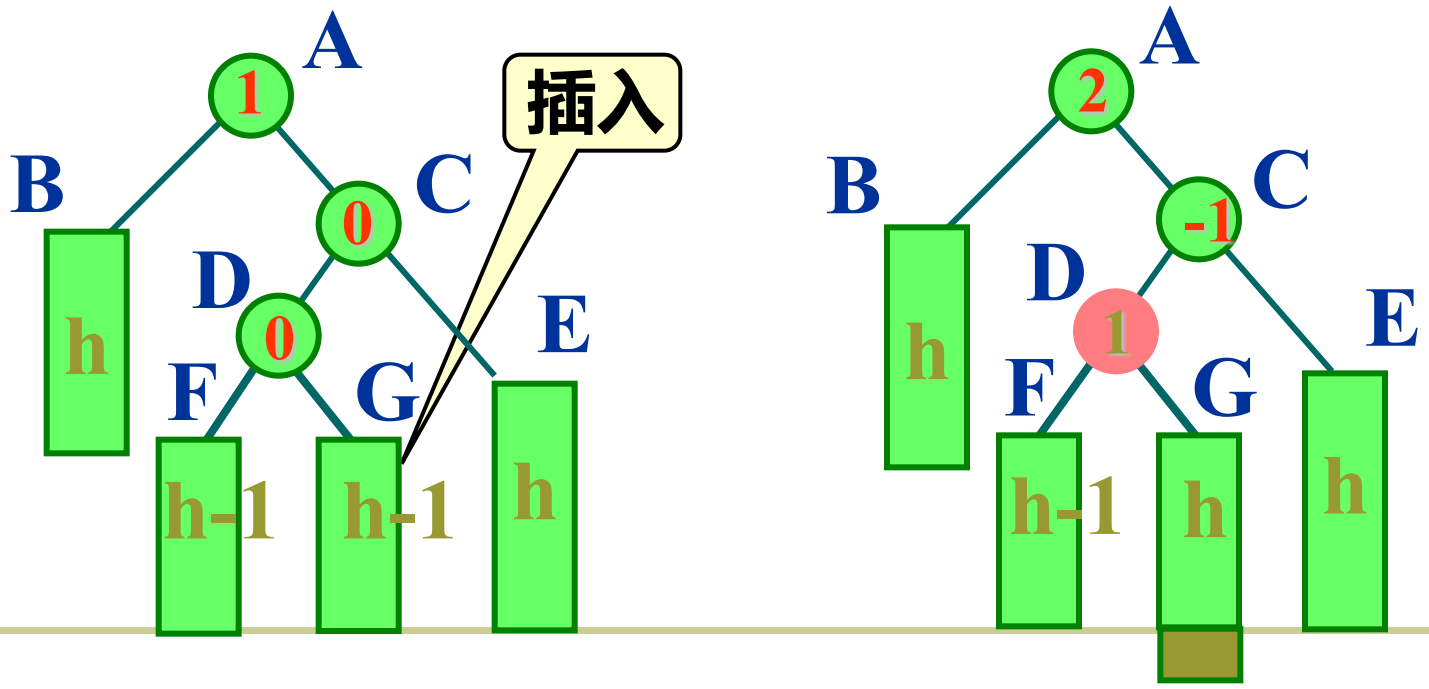
```
template <class K, class E>
void AVLTree<K, E>:: RotateLR (AVLNode<K, E> *& ptr) {
    AVLNode<K, E> *subR = ptr;
    AVLNode<K, E> *subL = subR->left;
    ptr = subL->right;
    subL->right = ptr->left;
    ptr->left = subL;
    if (ptr->bf <= 0) subL->bf = 0;
    else subL->bf = -1;
    subR->left = ptr->right;
    ptr->right = subR;
    if (ptr->bf == -1) subR->bf = 1;
    else subR->bf = 0;
    ptr->bf = 0;
};
```



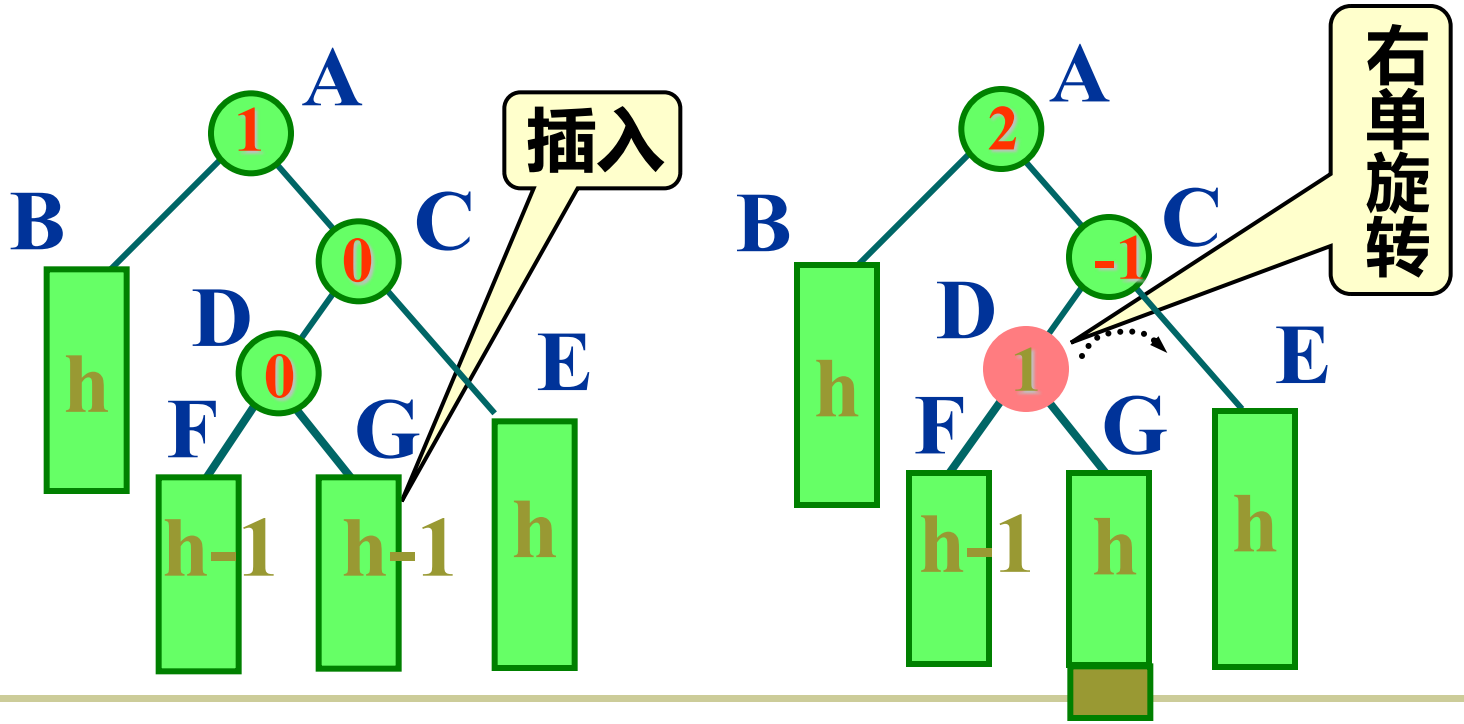


先右后左双旋转 (RotationRightLeft)

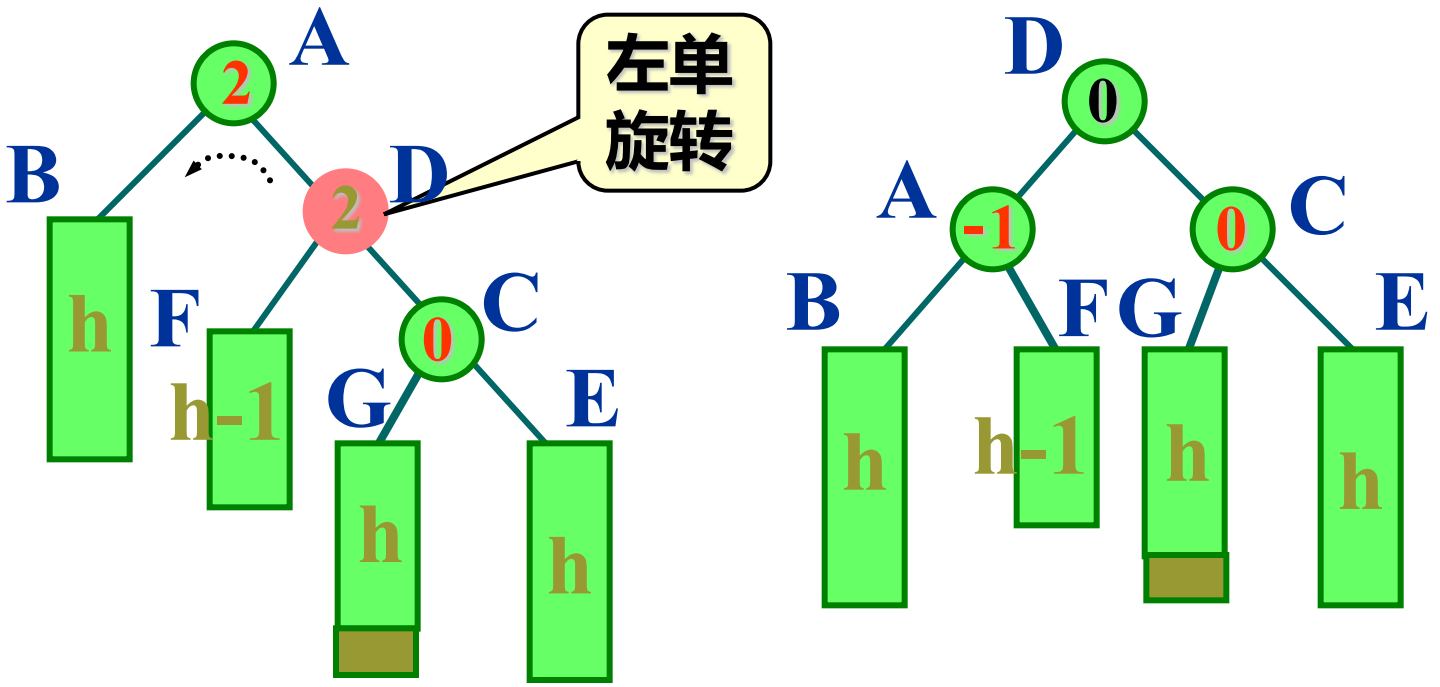
- 在结点A的右子女的左子树中插入新结点，该子树高度增1。结点A的平衡因子变为2，发生了不平衡。



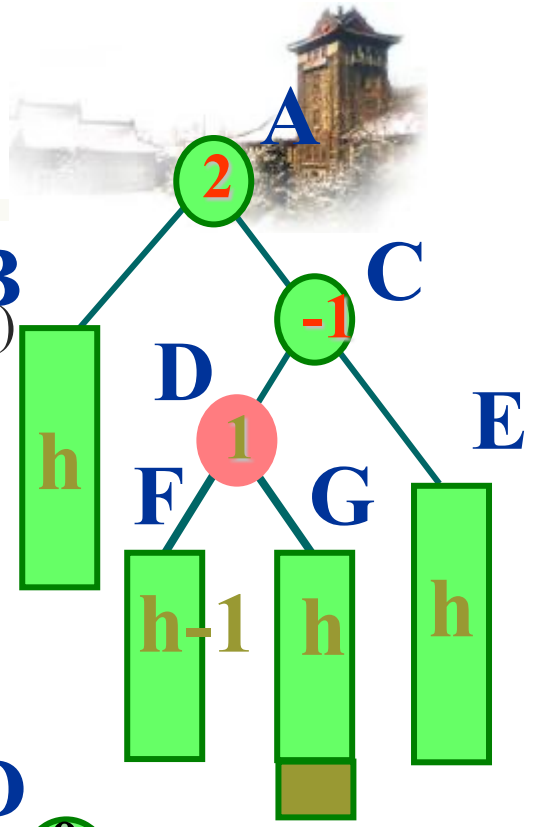
- 首先以结点D为旋转轴，将结点C顺时针旋转，以D代替原来C的位置。

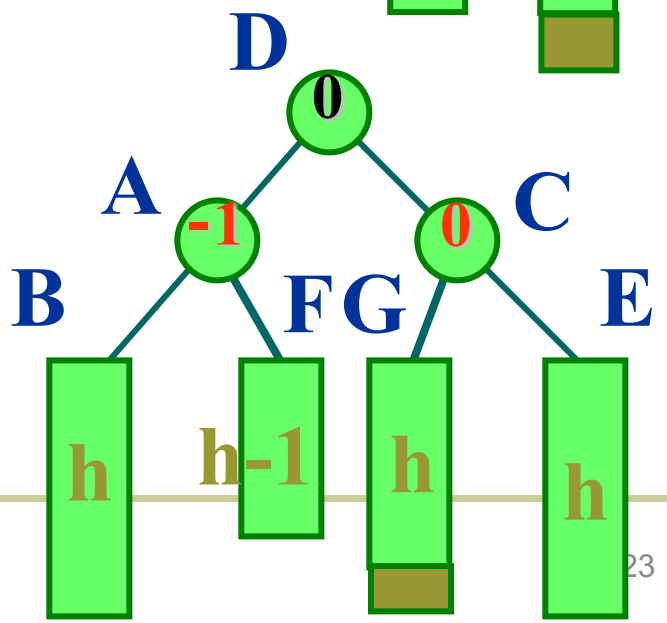


- 再以结点D为旋转轴，将结点A反时针旋转，恢复树的平衡。



```
template <class K, class E>
void AVLTree<K, E>::RotateRL (AVLNode<K, E> *& ptr)
 {
    AVLNode<K, E> *subL = ptr;
    AVLNode<K, E> *subR = subL->right;
    ptr = subR->left;
    subR->left = ptr->right;
    ptr->right = subR;
    if (ptr->bf >= 0) subR->bf = 0;
    else subR->bf = 1;
    subL->right = ptr->left;
    ptr->left = subL;
    if (ptr->bf == 1) subL->bf = -1;
    else subL->bf = 0;
    ptr->bf = 0;
};
```

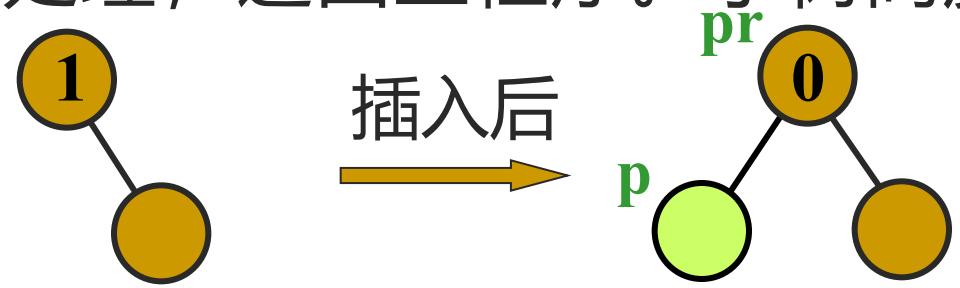




AVL树的插入

- 在插入新结点后，需从插入结点沿通向根的路径向上回溯。
- 如果在某一结点发现高度不平衡，停止回溯。从发生不平衡的结点起，沿刚才回溯的路径取直接下两层的结点。
  - 如果这三个结点处于一条直线上，则采用单旋转进行平衡化。
    - 单旋转可按其方向分为左单旋转和右单旋转。
  - 如果这三个结点处于一条折线上，则采用双旋转进行平衡化。
    - 双旋转分为先左后右和先右后左两类。

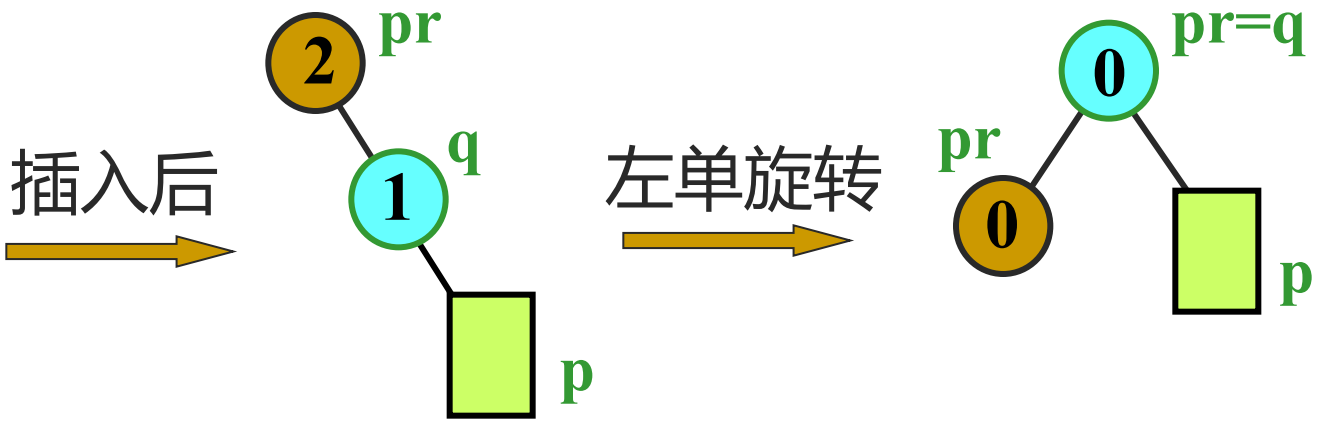
- 新结点p的平衡因子为0。设其父结点为pr，若p是pr的右子女，则pr的平衡因子增加1，否则pr的平衡因子减小1。
- 插入新结点并修改pr的平衡因子值后，pr的平衡因子值有三种情况：
  1. 结点pr的平衡因子为0。说明刚才是在pr的较矮的子树上插入了新结点，此时不需做平衡化处理，返回主程序。子树高度不变。



2. 结点pr的平衡因子的绝对值|bf| = 1。说明插入前pr的平衡因子是0，插入新结点后，以pr为根的子树不需平衡化旋转。但该子树高度增加，还需从结点pr向根方向回溯，继续考查结点pr双亲(pr = Parent(pr))的平衡状态。

3. 结点pr的平衡因子的绝对值|bf| = 2。说明新结点在较高的子树上插入，造成了不平衡，需要做平衡化旋转。此时可进一步分2种情况讨论：

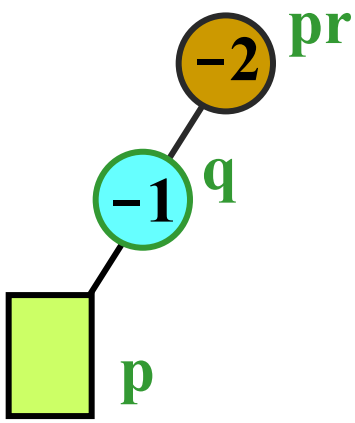
- 若结点pr的bf = 2，说明右子树高，结合其右子女q 的bf分别处理：
  - 若q的bf为1，执行左单旋转。



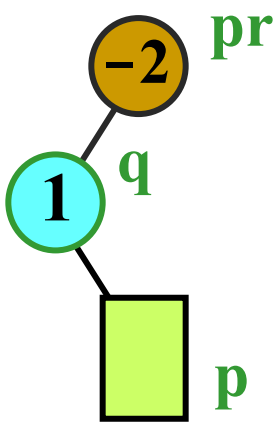
  - 若q的bf为-1，执行先右后左双旋转。

❷ 若结点pr的bf = −2，说明左子树高，结合其左子女q 的bf分别处理：

- 若q的bf为−1，执行右单旋转；
- 若q的bf为1，执行先左后右双旋转。



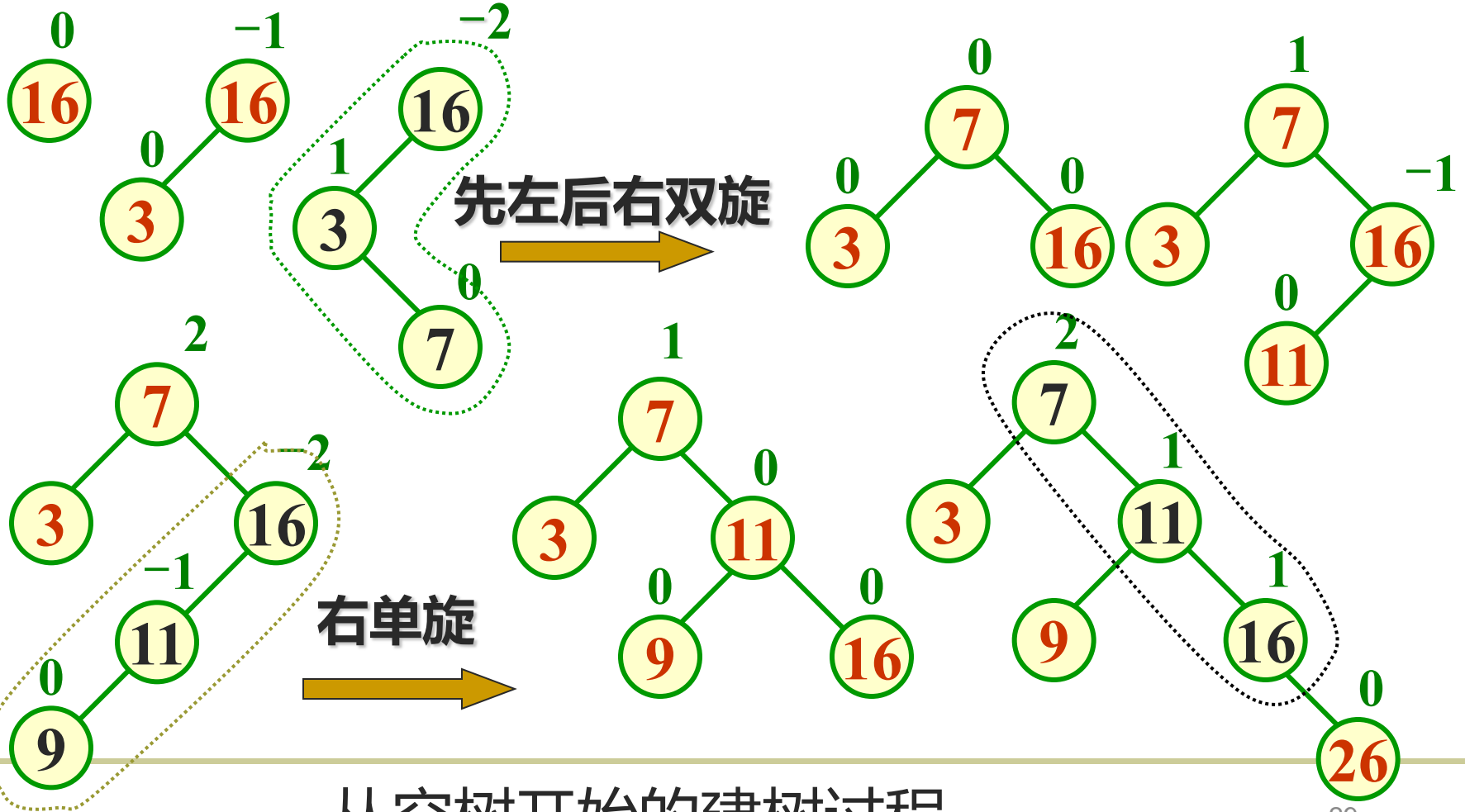
右单旋转



先左后右双旋转

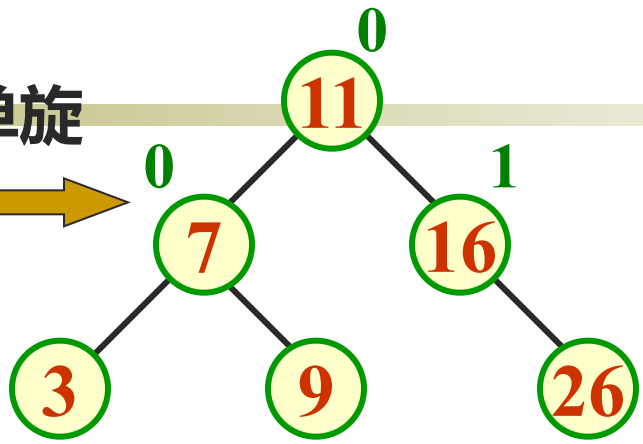
下面举例说明在AVL树上的插入过程

例如，输入关键码序列为 { 16, 3, 7, 11, 9, 26, 18, 14, 15 }，插入和调整过程如下。

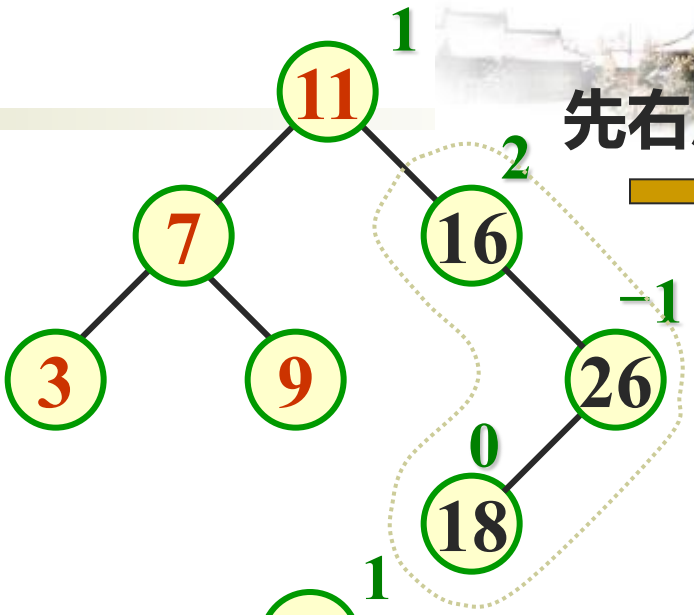


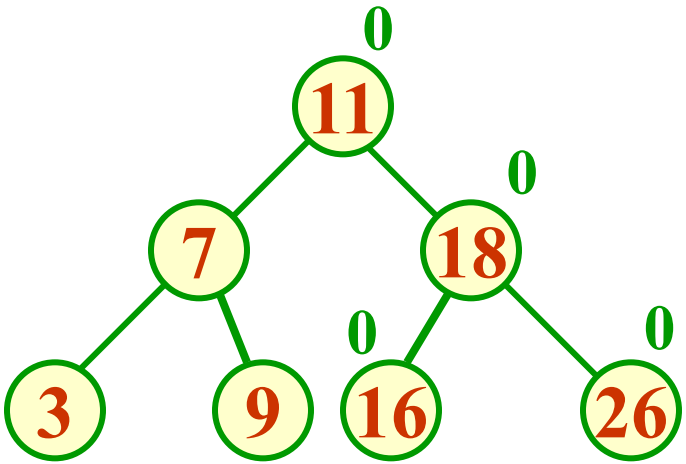
从空树开始的建树过程

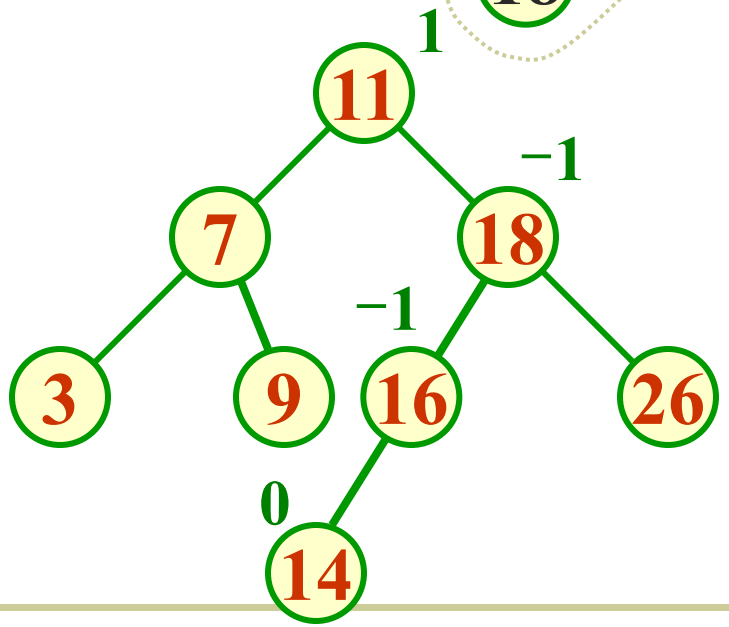
左单旋



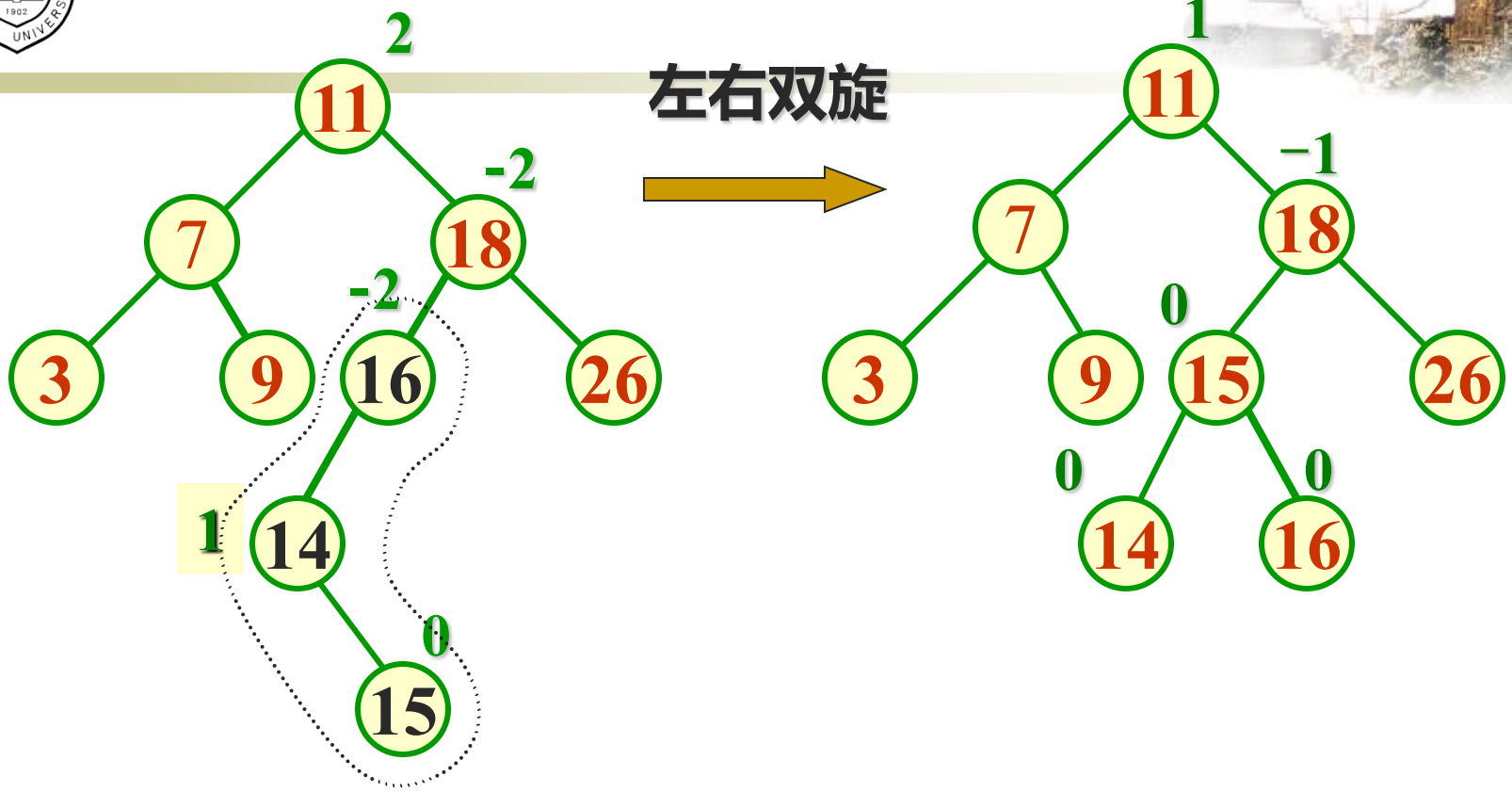
先右后左双旋







从空树开始的建树过程



从空树开始的建树过程

AVL树的删除

1. 如果被删结点x最多只有一个子女，可做简单删除:
   - 将结点x从树中删去。
   - 如果结点x有一个子女，可以简单地把x的双亲中原来指向x的指针改成指到这个子女结点;
   - 如果结点x没有子女，x双亲原来指向x的指针置为NULL。
   - 将原来以结点x的父结点为根的子树的高度减1。

2. 如果被删结点 x 有两个子女:
   - 搜索x在中序次序下的直接前驱 y (同样也可以找直接后继)。
   - 把结点 y 的内容传送给结点 x，现在问题转移到删除结点 y，把结点 y 当作被删结点。
   - 因为结点 y 最多有一个子女，可以简单地用 1. 给出的方法进行删除。

删除完x后，必须沿结点x的父结点通向根的路径反向追踪高度的变化对路径上各个结点的影响。

- 用一个布尔变量shorter（缩短）来指明子树高度是否被缩短。在每个结点上要做的操作取决于shorter的值和结点的bf，有时还要依赖子女的bf。
- 布尔变量shorter的值初始化为True。然后对于从x的双亲到根的路径上的各个结点p，如果shorter变成False，算法终止；在shorter保持为True时执行下面的操作：

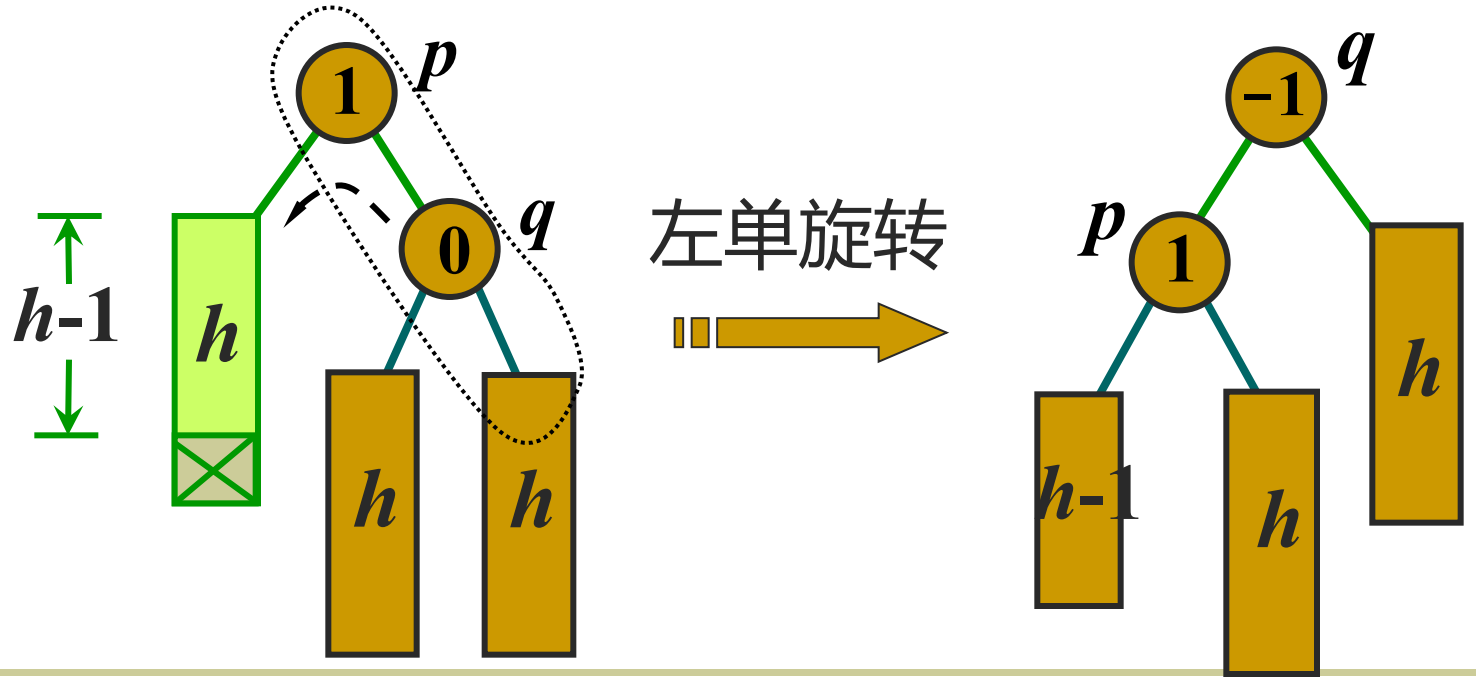
- 当前结点 p 的bf为0。如果它的左子树或右子树被缩短，则它的bf改为1或-1，同时 shorter置为False。

不需要继续检查上层结点的平衡因子

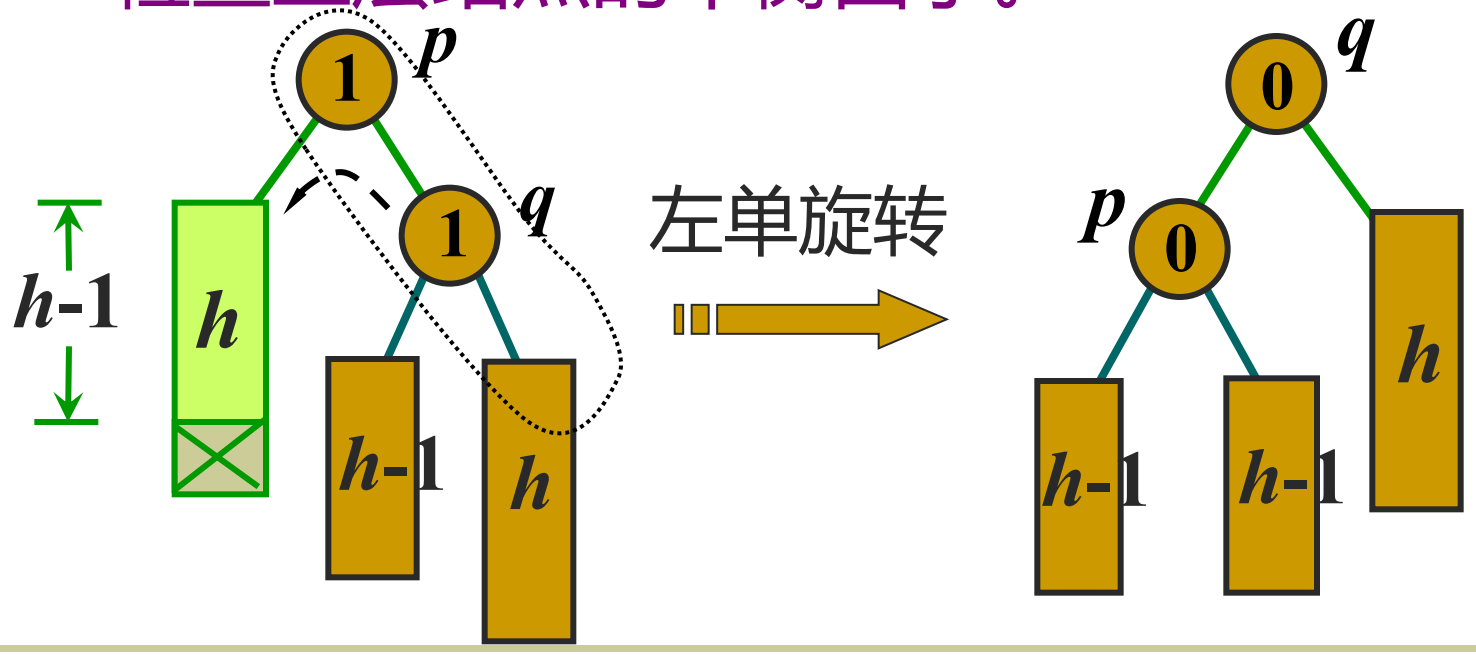
❷ 结点 p 的 bf 不为0且较高的子树被缩短，则 p 的 bf 改为0，同时shorter置为True。还要继续检查上层结点的平衡因子。

- ❸ 结点 p 的 bf 不为0，且较矮的子树又被缩短，则在结点 p 发生不平衡，需要进行平衡化旋转来恢复平衡。
  - 令 p 的较高的子树的根为 q（该子树未被缩短），根据 q 的 bf，有如下 3 种平衡化操作：

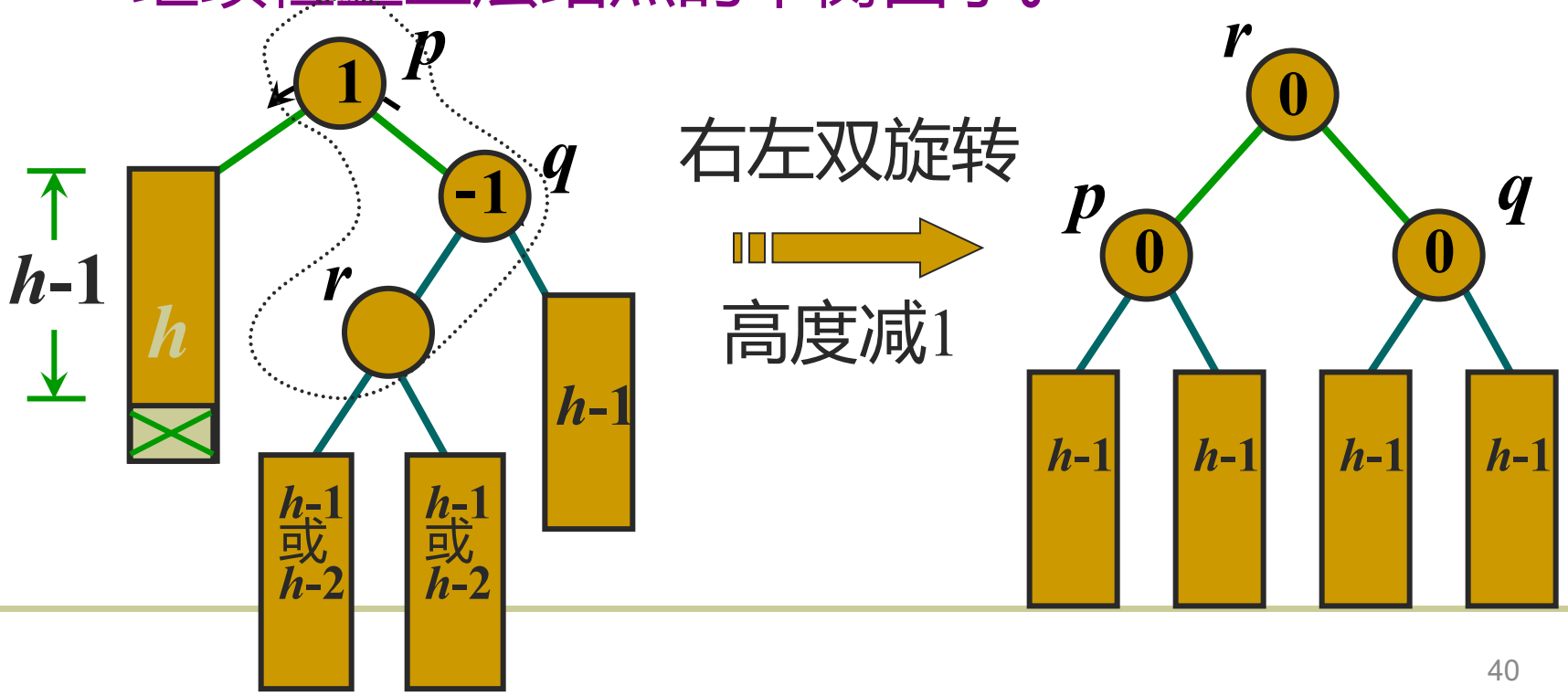
a) 如果 q（较高的子树）的 bf 为0，执行一个单旋转来恢复结点 p 的平衡，置shorter为False。无需检查上层结点的平衡因子。



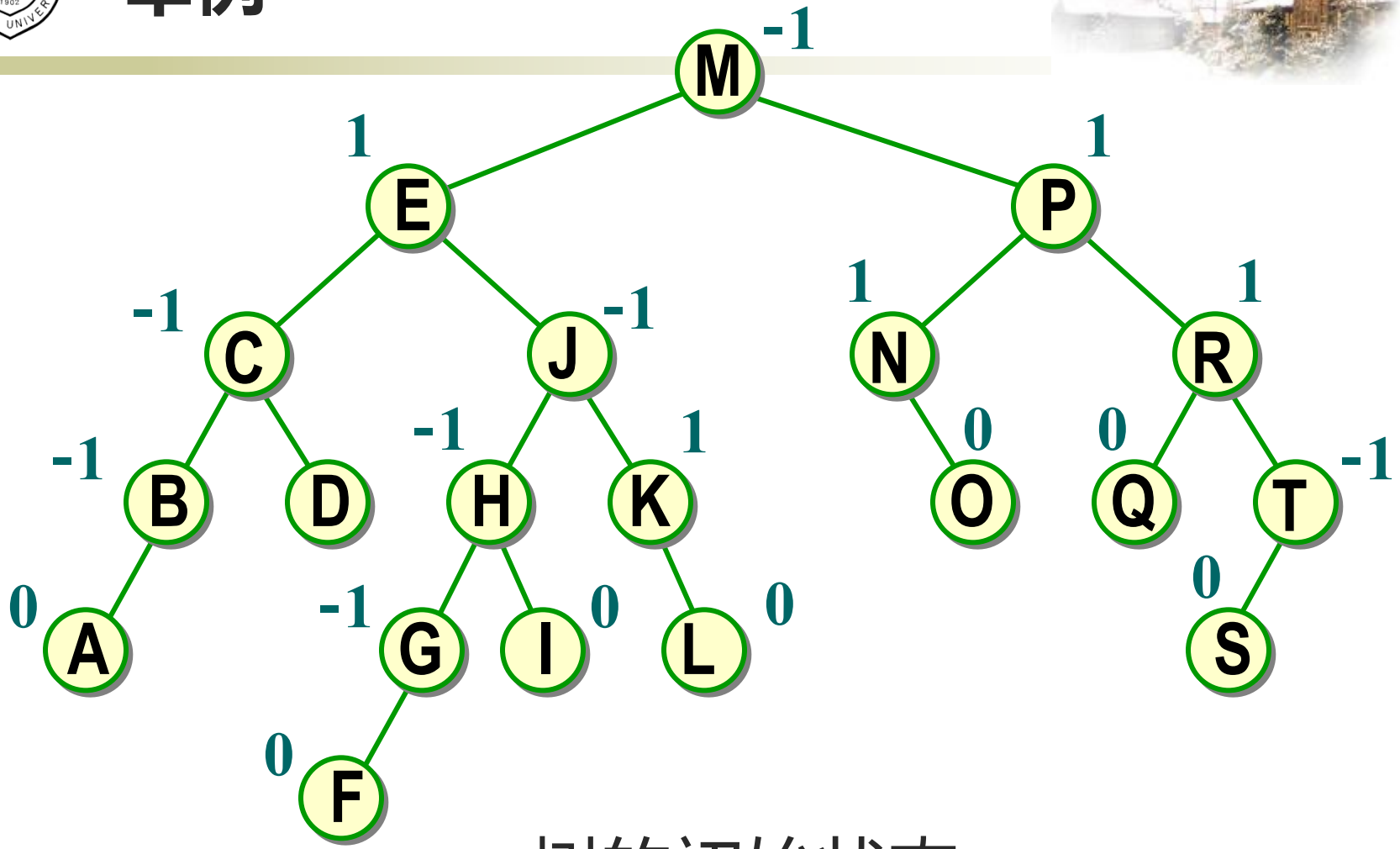
b) 如果 q 的 bf 与 p 的 bf 相同，则执行一个单旋转来恢复平衡，结点 p 和 q 的 bf 均改为0，同时置shorter为True。还要继续检查上层结点的平衡因子。



c) 如果 p 与 q 的 bf 相反，则执行一个双旋转来恢复平衡。新根结点的 bf 置为0，其他结点的 bf 相应处理，同时置shorter为True。还要继续检查上层结点的平衡因子。

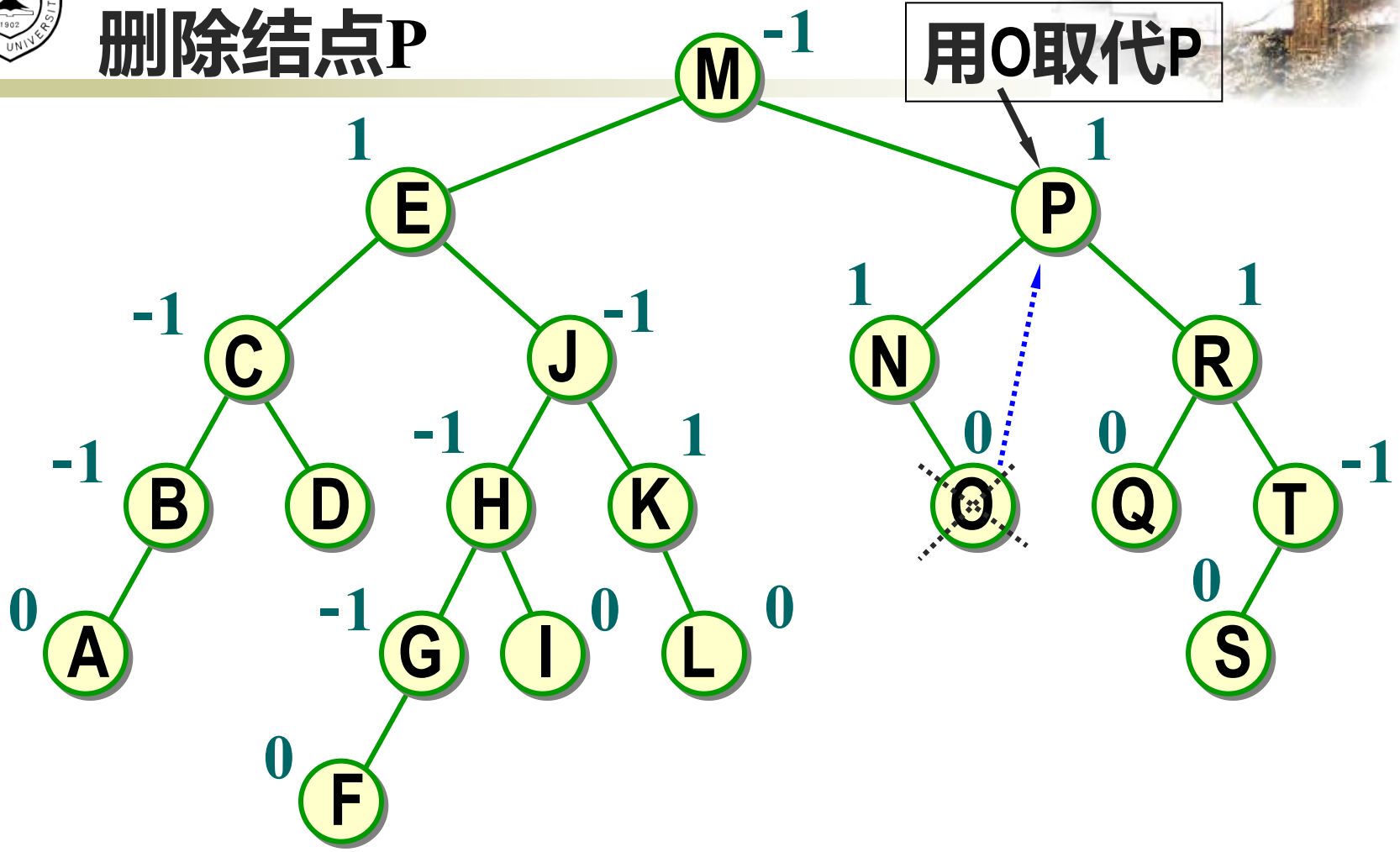


举例



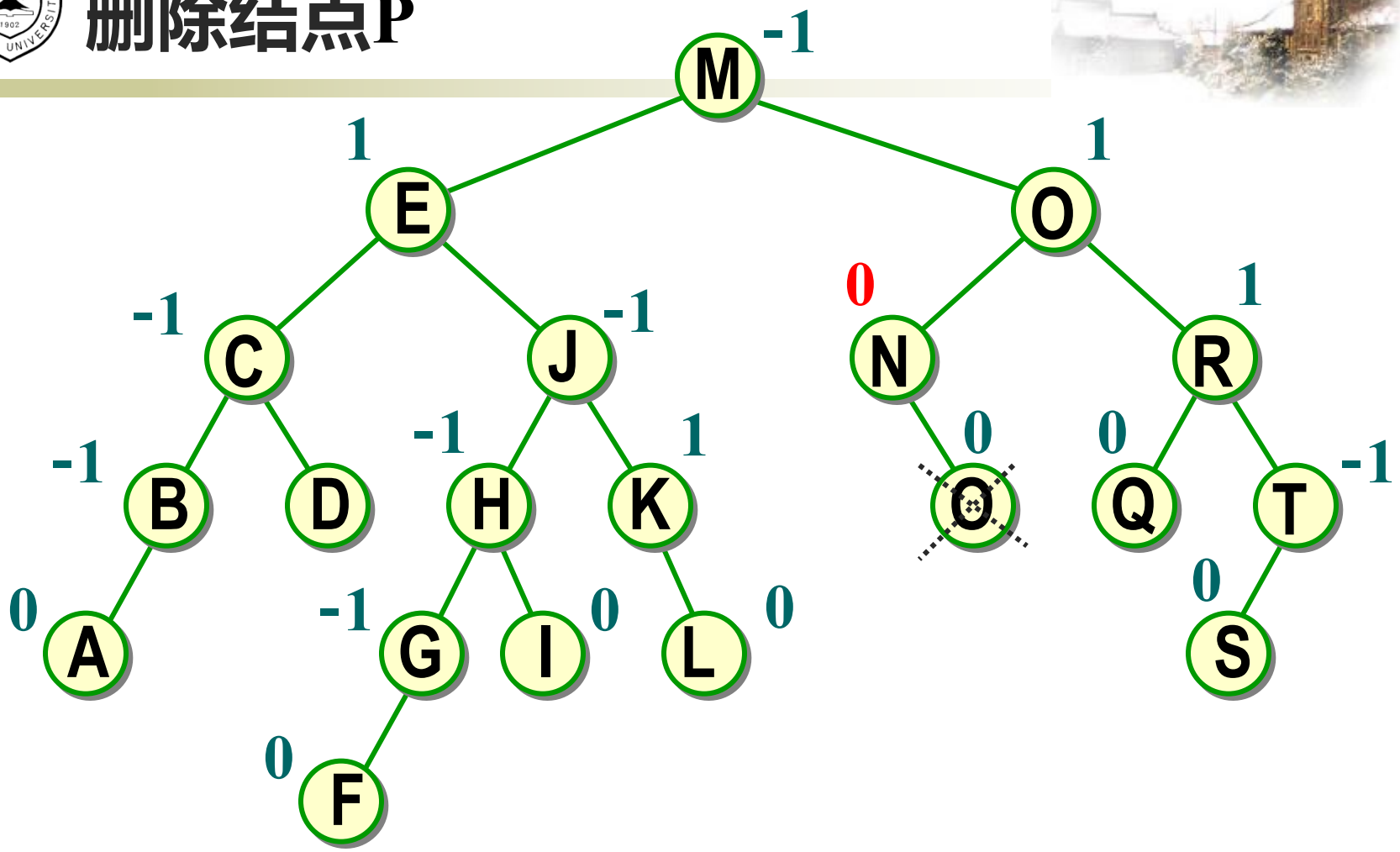
树的初始状态

删除结点P



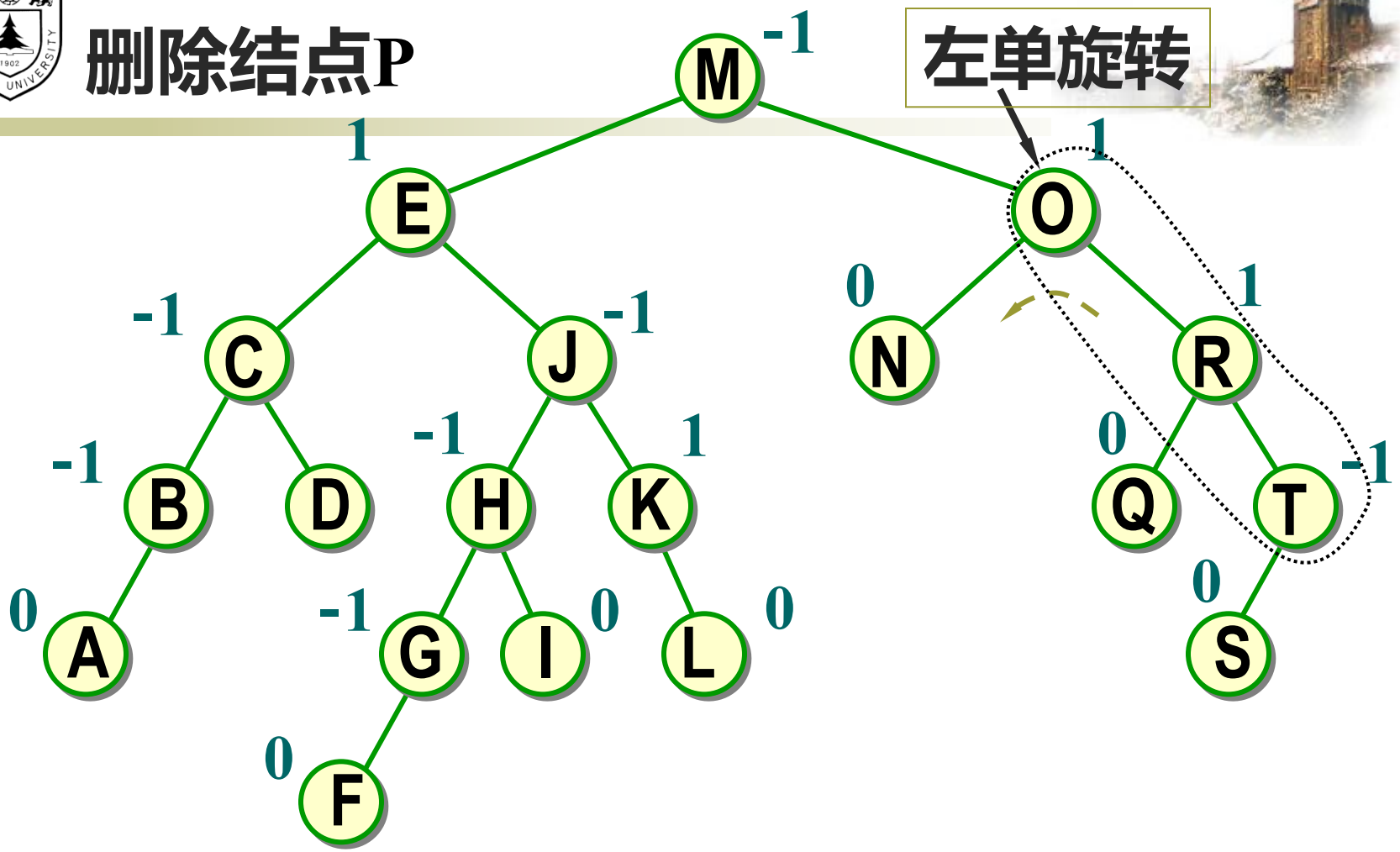
寻找结点P的中序直接前驱O, 用O顶替P, 删除O。

删除结点P



N的bf改为0，继续检查上层结点O。

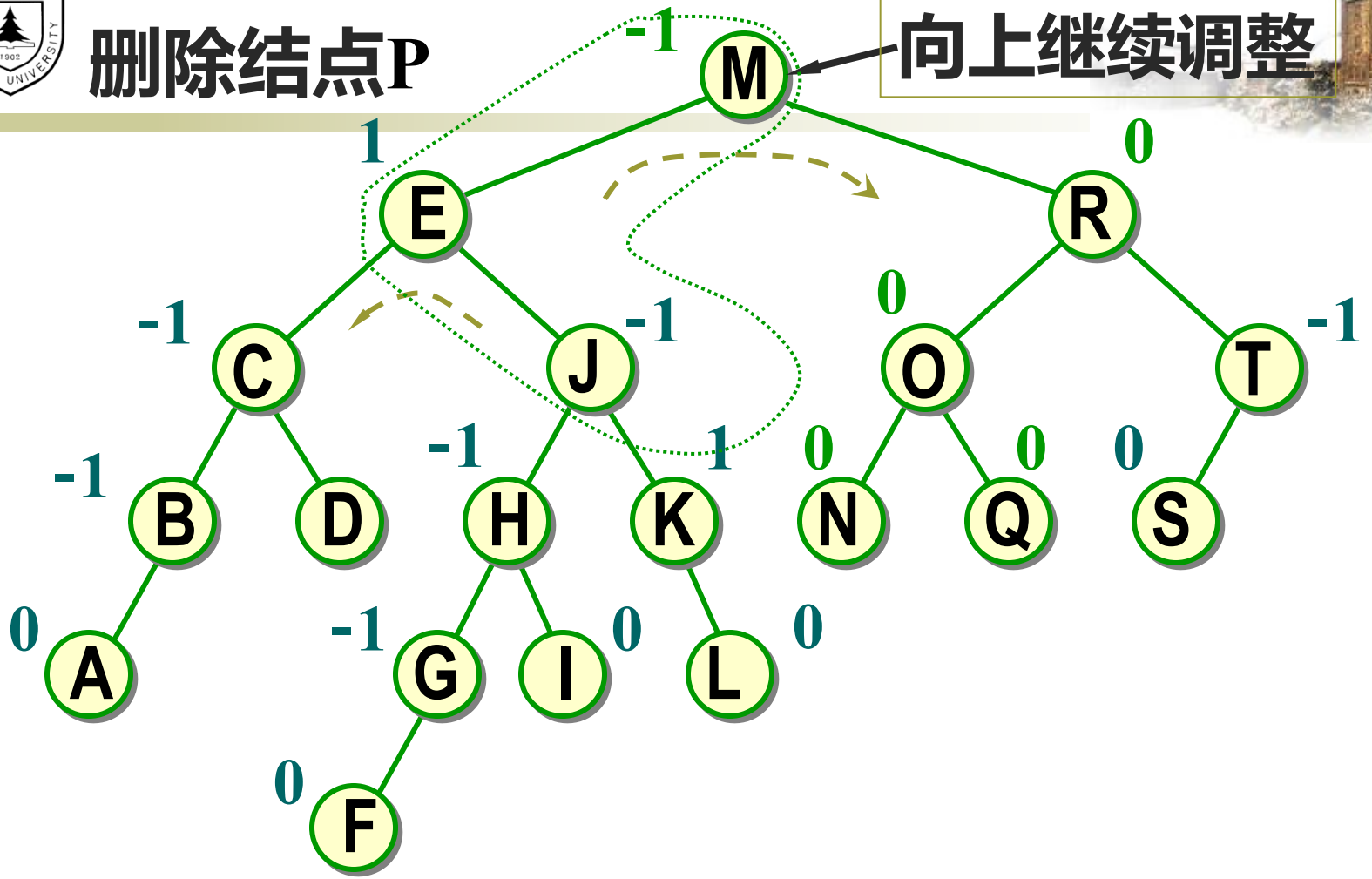
删除结点P



O与R的平衡因子同号，以R为旋转轴做左单旋转，M的子树高度减 1。

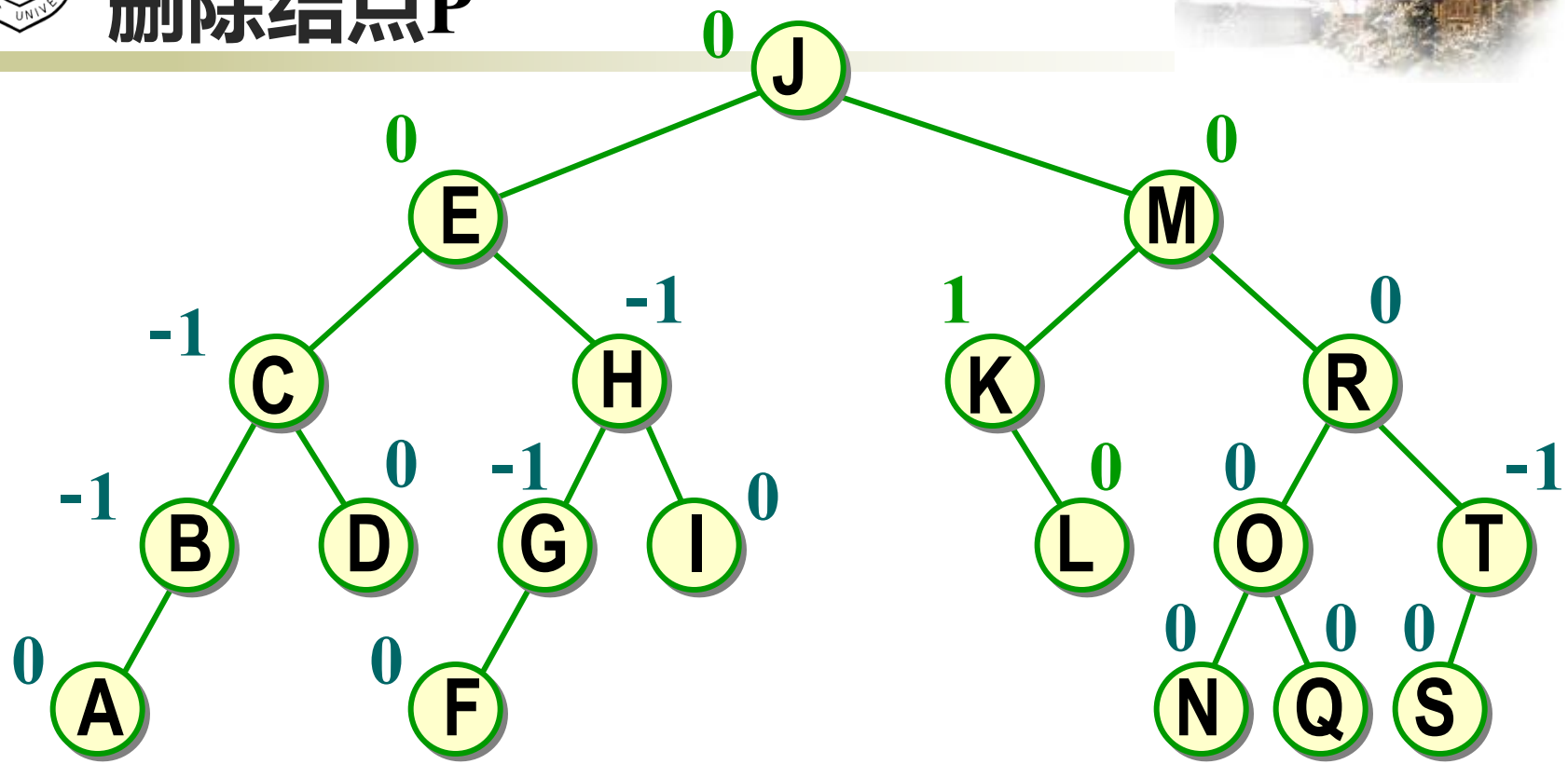
删除结点P

向上继续调整



M的子树高度减 1，M发生不平衡。M与E的平衡因子反号，做先左后右双旋转。

删除结点P



AVL树的性能分析

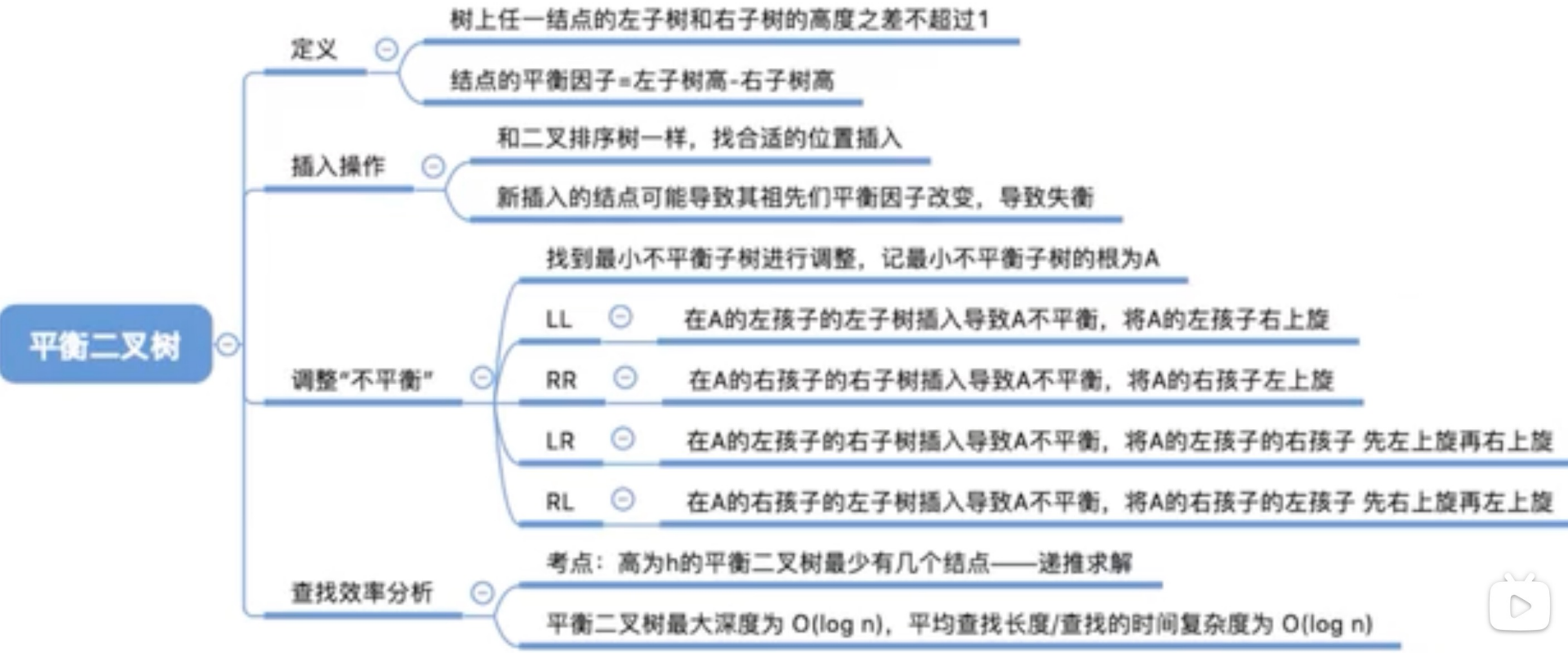
- 设在新结点插入前AVL树的高度为h，结点个数为n，则插入一个新结点的时间是O(h)。对于AVL树来说，h多大?
- 设 N_h 是高度为 h 的AVL树的最小结点数。最差情况下，根的一棵子树的高度为$h-1$，另一棵子树的高度为$h-2$，这两棵子树也是高度平衡的。因此有
  - $N_0 = 0$ (空树)
  - $N_1 = 1$ (仅有根结点)
  - $N_h = N_{h-1} + N_{h-2} + 1 \ , h > 1$ (类似斐波那契数列, $F_0 = 0, F_1 = 1, F_h = F_{h-1} + F_{h-2}$)

- 可以证明, 对于 $h \geq 0$, 有 $N_h = F_{h+2} - 1$ 成立。

- 由此可以推出，有 n 个结点的AVL树的高度不超过

  $1.44*\log_2(n+2)$

- 因此，在AVL树中，搜索、插入、删除一个结点并做平衡化旋转所需时间为 $O(\log_2 n)$。


平衡二叉树
定义
树上任一结点的左子树和右子树的高度之差不超过1
结点的平衡因子=左子树高-右子树高
插入操作
和二叉排序树一样，找合适的位置插入
新插入的结点可能导致其祖先们平衡因子改变，导致失衡
调整"不平衡"
找到最小不平衡子树进行调整，记最小不平衡子树的根为A
LL
在A的左孩子的左子树插入导致A不平衡，将A的左孩子右上旋
RR
在A的右孩子的右子树插入导致A不平衡，将A的右孩子左上旋
LR
在A的左孩子的右子树插入导致A不平衡，将A的左孩子的右孩子 先左上旋再右上旋
RL
在A的右孩子的左子树插入导致A不平衡，将A的右孩子的左孩子 先右上旋再左上旋
查找效率分析
考点：高为h的平衡二叉树最少有几个结点——递推求解
平衡二叉树最大深度为 O(log n)，平均查找长度/查找的时间复杂度为 O(log n)

提交时间：12 月1 日，上课前提交

- 第7 章：
- (1) 第342 页，7.2
- (2) 第342 页，7.3
- (3) 第342 页，7.8
- (4) 第343 页，7.15
- (5) 第343 页，7.16